

Code Audit Report

JohannaKatana/FAMILYCOURTCOMPANIONAI · 172 files · 16,125 lines · Generated Jun 2, 2026 17:07

PRODUCTION READINESS

BLOCKED

0/100

3

Critical blockers

Must fix before launch

5

High-severity

Fix in next sprint

OVERALL	SECURITY	PERFORMANCE	QUALITY
52/100	42/100	70/100	70/100

Severity breakdown

Security	3 critical · 5 high · 2 medium
Performance	No issues
Quality	No issues

Executive summary

This codebase is not safe to ship. The login screen grants access to anyone who clicks a button — no passwords checked, no identity verified. On top of that, any user can impersonate any other user by forging a single header value, giving them full access to other people's sensitive family court data. This app handles children's details, legal strategy, and witness information — the security gaps create serious legal and reputational exposure. Three critical blockers and five high-severity issues must be resolved before any public release.

BUSINESS RISKS

User Data Fully Exposed

Any person can access, modify, or delete another user's family court records — including children's details and case strategy — creating immediate legal liability and potential regulatory action under data protection laws.

No Real Authentication Exists

The sign-in screen is cosmetic only; anyone can enter the app without credentials, making every user's data accessible to any visitor and rendering the product unsellable to legal or enterprise clients.

Sensitive Data Stored Unprotected on Device

All case evidence and personal details sit unencrypted in the browser, readable by malicious browser extensions or anyone with physical device access, exposing the company to GDPR/CCPA violations and client lawsuits.

Automatic Database Changes on Code Merge

A single bad code merge can silently destroy production data with no rollback — one incident could cause irreversible data loss and catastrophic trust failure with users in active legal proceedings.

Cross-Origin Attack Window Open

A misconfigured CORS policy allows any website to make authenticated API calls on behalf of logged-in users, enabling account takeover attacks that could expose all case data.

TOP 5 PRIORITIES

CRITICAL Authentication via unverified client-supplied HTTP header (x-user-id)

artifacts/api-server/src/routes/cases.ts:10

Replace the forged-header trust model with a cryptographically verified JWT in a shared auth middleware so identity cannot be spoofed.

HIGH SignIn page performs no actual authentication — navigates directly on button click

artifacts/family-court-companion/src/pages/SignIn.tsx:62

Integrate a real authentication provider so sign-in validates credentials and issues a verified session before granting any access to the application.

CRITICAL Mass-assignment vulnerability in PUT /cases/:id and PUT evidence/:id

artifacts/api-server/src/routes/cases.ts:41

Validate and allowlist request body fields before saving to the database so users cannot overwrite ownership or system fields.

CRITICAL Sensitive case data persisted in localStorage without encryption

artifacts/family-court-companion/src/lib/dataService.ts:1

Move sensitive data storage to the authenticated server-side database, or encrypt it with a user-derived key before writing to the browser.

HIGH CORS configured to allow any origin when ALLOWED_ORIGIN is unset

artifacts/api-server/src/app.ts:30

Remove the open-fallback setting and require an explicit allowed-origin value at server startup, failing fast if it is missing.

RECOMMENDED NEXT STEPS

1. Build and integrate real authentication (JWT or a managed auth provider) — fix the sign-in bypass and the forged-header identity issue at the same time.

Fix the mass-assignment vulnerability so API update endpoints only accept explicitly allowlisted fields, preventing ownership hijacking.

3. Stop storing sensitive case data unencrypted in the browser — migrate persistence to the server-side database behind the authenticated API.
4. Lock down the CORS configuration to an explicit origin allowlist with no open fallback.
5. Replace the automatic database schema push on git merge with a reviewed, manual migration step in the deployment pipeline.
6. Add database foreign-key constraints and indexes on case ID columns to enforce data integrity and prevent future performance degradation.
7. Enable Content Security Policy headers and tighten rate limits on write and authentication endpoints before any public launch.

Security

CRITICAL Authentication via unverified client-supplied HTTP header (x-user-id)

artifacts/api-server/src/routes/cases.ts:10

Every route in `cases.ts`, `evidence.ts`, and `scans.ts` authenticates by reading `req.headers['x-user-id']` directly. Any caller can forge this header to impersonate any user. There is no JWT validation, session cookie verification, or middleware that cryptographically proves identity. This means any authenticated-looking request can read, modify, or delete another user's sensitive family-court data.

Suggested fix: Replace the `x-user-id` header trust model with a cryptographically verified token (e.g. verify a JWT in a dedicated auth middleware applied at the router group level). The middleware should extract and attach a verified `userId` to `req.user`, and handlers should consume `req.user.id` — never a raw header.

CRITICAL Mass-assignment vulnerability in PUT /cases/:id and PUT evidence/:id

artifacts/api-server/src/routes/cases.ts:41

In `cases.ts` line ~41, the update handler does `db.update(casesTable).set({ ...req.body, updatedAt: new Date() })` without any allowlist or schema validation. An attacker can overwrite any column including `userId`, effectively hijacking ownership of the case. The same pattern exists in `evidence.ts` for the PUT route.

Suggested fix: Parse `req.body` through the `insertCaseSchema` (or a dedicated `updateCaseSchema` that omits `id/userId/createdAt`) before passing to `db.update()`. Only spread `parsed.data` into the `set()` call, just as the POST handler does. Apply identical fix to `evidence.ts` PUT route.

CRITICAL Sensitive case data (names, children details, quotes) persisted in localStorage without encryption

artifacts/family-court-companion/src/lib/dataService.ts:1

`dataService.ts` stores all evidence entries, communication scans, and case preferences in `localStorage` under a plain-text namespace. The app handles highly sensitive family-court data — direct quotes, witness names, children's medical details, and case strategy — all readable by any JavaScript running on the same origin (XSS), browser extensions, and any person with physical access to the device.

Suggested fix: For truly sensitive data, encrypt before writing to `localStorage` (e.g. using a user-derived key from `SubtleCrypto`) or migrate persistence to the authenticated backend API so data is stored in the server-side PostgreSQL. At minimum, add a clear warning to the user that `localStorage` is not encrypted on-device.

HIGH CORS configured to allow any origin when ALLOWED_ORIGIN is unset

artifacts/api-server/src/app.ts:30

In `app.ts`, `cors({ origin: process.env.ALLOWED\_ORIGIN ?? true })` — when the env var is absent, `true` allows any origin to make credentialed cross-origin requests. Because `credentials: true` is also set, this allows a malicious site to make authenticated API calls on behalf of a user whose cookies or tokens are attached by the browser.

Suggested fix: Remove the `?? true` fallback. Require ALLOWED_ORIGIN to be explicitly configured. Fail fast at startup if it is absent (similar to how PORT is handled in index.ts). Use an explicit allowlist array for multiple origins if needed.

HIGH Missing database foreign-key constraints — evidence and scans can be orphaned or cross-case

lib/db/src/schema/evidence.ts:7

In `evidence.ts` and `scans.ts` schemas, `caseId` is defined as `uuid('case\_id').notNull()` but there is no `.references(() => casesTable.id)` foreign-key declaration. This means evidence records can reference non-existent or deleted cases, and — because the authorization check (`verifyCaseOwner`) fetches by caseId separately before the DML — a race condition could allow an attacker to insert evidence into a case they don't own if the case is deleted between the two queries.

Suggested fix: Add `.references(() => casesTable.id, { onDelete: 'cascade' })` to the caseId column in both evidence.ts and scans.ts schemas. Run a migration. This enforces referential integrity at the database level and eliminates the race window.

HIGH No indexes on caseId foreign keys — full table scans on every evidence/scan query

lib/db/src/schema/evidence.ts:1

The evidenceTable and communicationScansTable schemas have no index declarations on the `case\_id` column. Every `WHERE case\_id = ?` query (the most common query in the application) will perform a sequential table scan. As cases accumulate evidence, this will degrade to O(n) per request.

Suggested fix: Add an index in both schemas: `index('evidence\_case\_id\_idx').on(evidenceTable.caseId)` and similarly for scans. Also add a composite index on `(userId)` for the casesTable since the GET /cases query filters by userId.

HIGH SignIn page performs no actual authentication — navigates directly on button click

artifacts/family-court-companion/src/pages/SignIn.tsx:62

The Sign In button in `SignIn.tsx` calls `setLocation('/home')` without any credential validation, API call, or auth state update. Any user who clicks 'Sign In' is immediately granted access to the full application regardless of what they typed. The Sign Up page similarly calls `setLocation('/onboarding/state')` without creating an account.

Suggested fix: Integrate a real auth provider (Clerk, Supabase Auth, Firebase Auth, or custom JWT flow). The sign-in button should call an auth API, await a verified session token, store it securely (httpOnly cookie or memory, not localStorage), and only then navigate. Protect all post-auth routes with an auth guard component.

HIGH Rate limiter applies globally — no per-user or per-endpoint limits for sensitive write operations

artifacts/api-server/src/middlewares/security.ts:8

The rate limiter in `security.ts` allows 200 requests per 15 minutes globally per IP. There are no tighter limits on mutation endpoints (POST evidence, POST scans) or on the authentication endpoint. An attacker behind a residential IP can make 200 requests in 15 minutes without restriction, which is sufficient for bulk data scraping or brute-force if auth is added later.

Suggested fix: Apply tighter rate limits to write endpoints (e.g. 30 req/15 min for POST/PUT/DELETE) and even tighter limits for any future authentication endpoints (e.g. 10 req/15 min). Use express-rate-limit at the route group level with a stricter config.

MEDIUM **Content-Security-Policy is fully disabled**

artifacts/api-server/src/middlewares/security.ts:7

In `security.ts`, `contentSecurityPolicy: false` is passed to Helmet. CSP is the primary browser defense against XSS — disabling it means any XSS vulnerability can exfiltrate localStorage data (which contains all case evidence) or make arbitrary API calls.

Suggested fix: Configure a restrictive CSP rather than disabling it. At minimum set `default-src 'self'` and only allowlist specific origins needed by the app. For the React SPA served separately, add a CSP meta tag or configure the Vite dev server / production CDN to serve the appropriate CSP header.

MEDIUM **scripts/post-merge.sh runs drizzle-kit push (schema push) automatically on every git merge**

scripts/post-merge.sh:3

`pnpm --filter db push` in post-merge.sh runs `drizzle-kit push --config` which applies schema changes directly to the production database without a migration review step. If a destructive schema change is merged (e.g. column rename or drop), it will be applied to production automatically with no rollback plan.

Suggested fix: Replace drizzle-kit push with drizzle-kit generate + a controlled migration application step (drizzle-kit migrate) as part of a CI/CD pipeline with a review gate. Never run schema mutations automatically on post-merge in a production workflow.

Performance

No issues detected.

Code Quality

No issues detected.